

# Regression basics

Marcin Włodarczak

25 October 2024

As usual, we start by loading `tidyverse`.

```
library('tidyverse')  
theme_set(theme_bw())
```

In addition, we will need a couple of helper function I wrote for this class and saved in the file `04-lab-utils.R`. We can load them like this (**remember to set the working directory to the location of the present file first: Session -> Set Working Directory -> To Source File Location**):

```
source('04-lab-utils.R')
```

Finally, we will be using a data set from the `languageR` package, which we need to install. Remember that you only need to do it once.

```
install.packages('languageR')
```

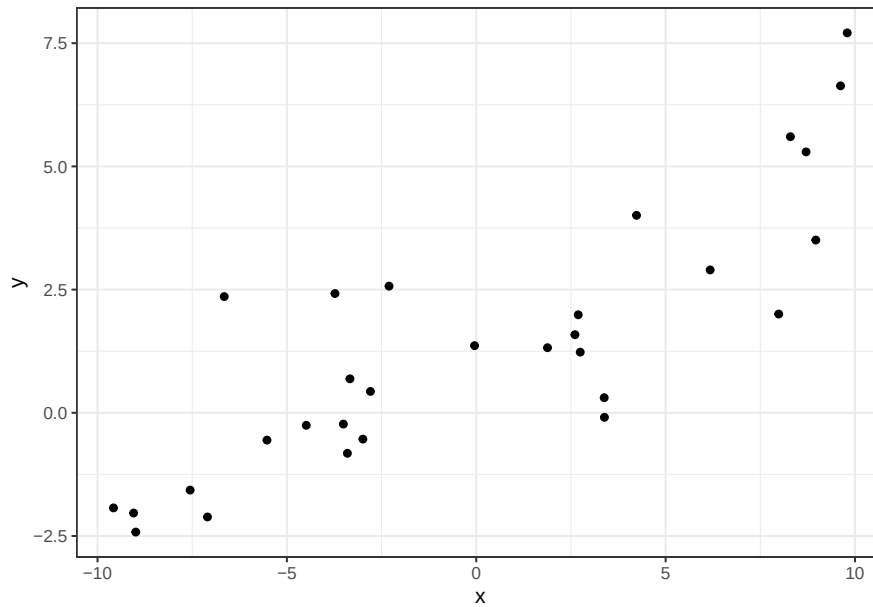
## Bivariate data

Let's start by generating some bivariate data. Our independent variable (`x`) consists of 30 random numbers between -10 and 10. To construct the dependent variable (`y`), we multiply each of these points by 0.5 and add 1. Finally, we add some normally distributed noise with the mean of 0 and standard deviation of 2.

```
d <- tibble(x = runif(30, -10, 10),  
            y = 1 + 0.5 * x + rnorm(length(x), 0, 2))
```

We can assess the relationship between the variables using a scatterplot:

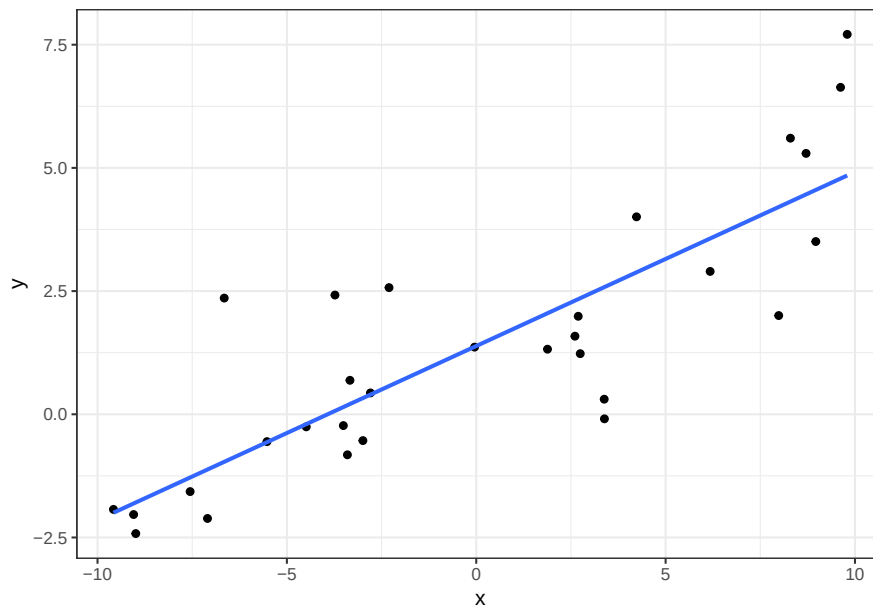
```
ggplot(d, aes(x, y)) +  
  geom_point()
```



We can add a straight line to the plot using `geom_smooth` with the `lm` method. By default, `geom_smooth` will draw a confidence interval (more on this later in the course) around the line. We do not want it for now so we switch it off by setting `se` to `FALSE`.

```
ggplot(d, aes(x, y)) +  
  geom_point() +  
  geom_smooth(method = 'lm', se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



In order to actually fit a regression model, we need to use the `lm` function. The first argument is a *formula*, which specifies the dependent variable (`y`) and the independent variables (we only have one, `x`), separated with a tilde. The second argument is the data frame (or tibble) where the columns named in the formula can be found.

```
m <- lm(y ~ x, d)
```

To summarise, what we did above was fit a simple regression model with  $y$  as a dependent and  $x$  as an independent variable and assigned it to variable  $m$ . Let's now print the model summary:

```
summary(m)

##
## Call:
## lm(formula = y ~ x, data = d)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.6732 -0.8294 -0.2549  0.7447  3.3232
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.38567    0.26864   5.158 1.80e-05 ***
## x            0.35324    0.04418   7.995 1.05e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.471 on 28 degrees of freedom
## Multiple R-squared:  0.6954, Adjusted R-squared:  0.6845
## F-statistic: 63.92 on 1 and 28 DF,  p-value: 1.046e-08
```

## Slope and intercept

There is a lot information in the output above but for now we will concentrate on “Estimate” column in the “Coefficients” section. We can extract them like this:

```
coefs <- coefficients(m)
```

One thing to notice is that they are quite close to the values we supplied when we created the data ( $y = 1 + 0.5 * x$ ). In other words, the coefficients of the model correspond to the process which generated the data in the first place. It works! Notice also that the values are not exactly identical because we added some random (normally distributed) noise with the mean of 0 and the standard deviation of 2: `rnorm(length(x), 0, 2)`.

Let's assign the coefficients to variables:

```
b0 <- coefs[1] # Intercept
b1 <- coefs[2] # Slope
```

We can indeed see that slope correspond to a one unit increase:

```
y1 <- b0 + b1 * 5 # Value for x = 5
y2 <- b0 + b1 * 6 # Value for x = 6
y2 - y1
```

```
## (Intercept)
##      0.353244
```

Trivially, the intercept corresponds to the value of  $y$  when  $x$  equals 0:

```
b0 + b1 * 0 # Value for x = 0
```

```
## (Intercept)
##      1.385668
```

## Exercise

Generate data with different values of slope and intercept and refit the model. Make sure you understand what is going on.

Next, play with the `rnorm(length(x), 0, 2)` term above by changing the last value, which controls how much noise there is. Observe how the coefficients get (on average) increasingly close to the target values as you reduce the noise.

## Real-life example

As a real-life example of regression analysis, let's look at the `beginningReaders` data set from the `languageR` package:

```
library('languageR')
data('beginningReaders')
```

This data set consists of reaction times (in  $\log(s)$ , more on this next time) from a bunch of eight-year-old Dutch children elicited in a lexical decision task. Again, there are lots of columns (for details see `?beginningReaders`) but we will use the following:

- **LogRT**: reaction time (in  $\log(s)$ )
- **Subject**: subject ID
- **ReadingScore**: a score of reading proficiency

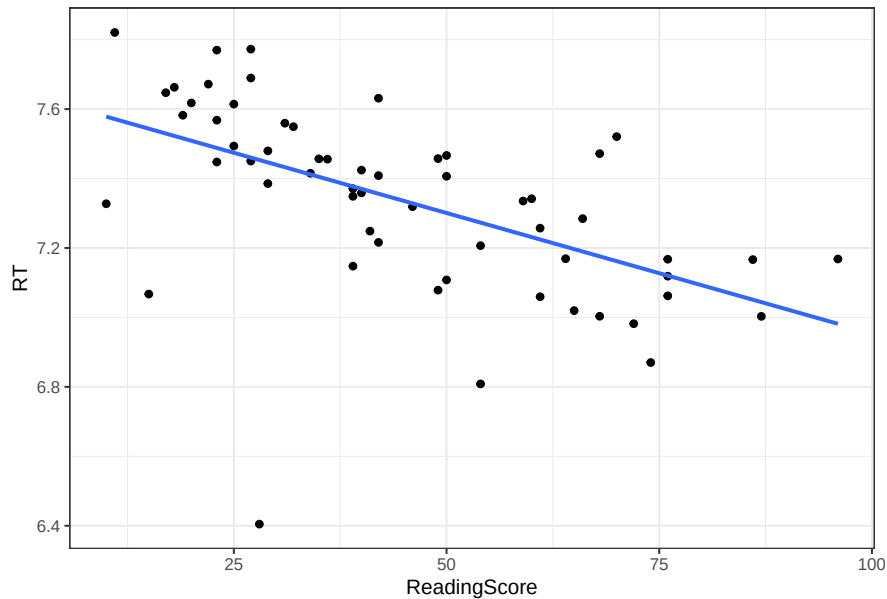
We want to see whether children with higher proficiency have lower reaction times. Before we can do that, however, we need to calculate average reaction time for each child. Since the value of **ReadingScore** is the same withing a subject, we just store the first value with `first`.

```
readers <- beginningReaders %>%
  group_by(Subject) %>%
  summarise(RT = mean(LogRT),
            ReadingScore = first(ReadingScore))
```

Now, we can visualise the relationship:

```
ggplot(readers, aes(ReadingScore, RT)) +
  geom_point() +
  geom_smooth(method = 'lm', se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



This does not look too bad but notice that there were a couple of kids with low reading proficiency who were nevertheless rather fast. This is potentially problematic (such values could have a disproportionately large influence on the slope of the regression line) but we won't worry about it for now.

Let's fit a linear model and look at the coefficients:

```
m_readers <- lm(RT ~ ReadingScore, readers)
coefficients(m_readers)
```

```
## (Intercept) ReadingScore
## 7.647536473 -0.006936342
```

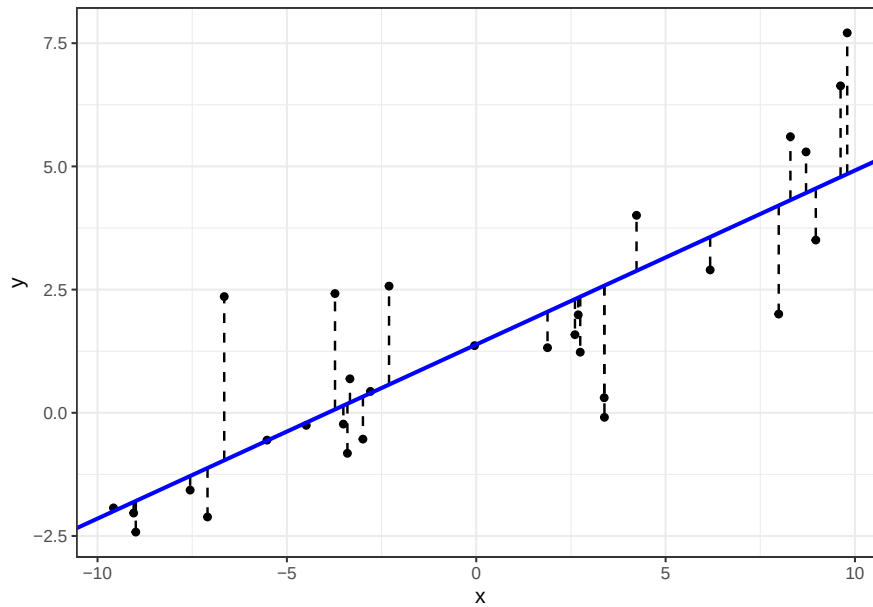
Here is how we would interpret the coefficients:

- **Slope:** the value of slope is the change in the dependent variable (RT) corresponding to the change of one in the independent variable. In our case, this means that an increase of one point on the reading score reduces the reaction time by -0.007 log(s).
- **Intercept:** the intercept gives us the expected value of the dependent variable when the independent variable is equal to 0. Here, this would be the reaction time of a child with a reading score of 0 (i.e. a child who cannot read at all), which is quite meaningless. The problem of intercept being impossible to interpret is quite common and we will see what can be done about it next time.

## Residuals

Coming back to our simulated data, because it includes some random noise, the data points do not lie exactly on the regression line. The difference between the actual data point and its estimate (i.e. the corresponding value lying on the regression line) is called the residual. We can visualise the residuals with the `plot_resid` function:

```
plot_resid(d$x, d$y)
```



Note that `plot_resid` is a function defined in the file `04-lab-utils.R` so you have to load that file first.

We can also extract residuals directly from the model using the `residuals()` function:

```
res <- residuals(m)
```

Now we can also decipher the “Residuals” section of the model summary. R is simply giving us a little summary of how the residuals are distributed. For instance, we can calculate the median value and compare it to the value in the model summary:

```
median(res)
```

```
## [1] -0.25494
```

Distribution of residuals plays an important role in the process of making sure our model is an adequate description of the data. We will come back to it later.

## Exercise

Obtain the other residual statistics from the model summary:

```
# Write your solution here.
```

## Exercise

Plot residuals for the `beginningReaders` model we constructed above.

```
# Write your solution here.
```

## Goodness of fit

As a measure of how well this model describes the data, we can calculate the residual sum of squares – we square the residuals (to get rid of the negative values) and sum them all up:

```
sum_sq <- sum(res^2)
```

This value tells how much variation in the data is left unexplained (or unaccounted for) in the model. In our case, we get a value of 60.621. Is this much? To be able to answer this question, we need some baseline

to compare our model against. For this purpose, we can use an intercept-only model, that is a model where the estimated value of  $y$  does not depend on  $x$ . We can specify such a model using 1 as a symbol for the intercept in the formula:

```
m0 <- lm(y ~ 1, d)
```

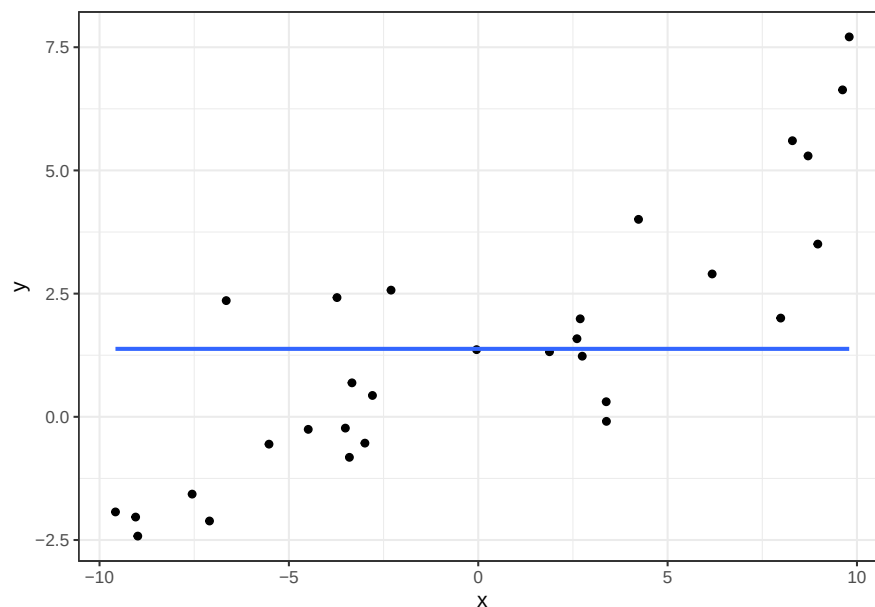
Notice that the model summary does not include the slope:

```
summary(m0)
```

```
##
## Call:
## lm(formula = y ~ 1, data = d)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.7994 -1.8432 -0.1033  1.1542  6.3309
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.3789     0.4783   2.883  0.00735 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.62 on 29 degrees of freedom
```

Let's plot the regression line (we specify the formula we want,  $y \sim 1$ , using an argument to `geom_smooth`):

```
ggplot(d, aes(x, y)) +
  geom_point() +
  geom_smooth(method = 'lm', se = FALSE, formula = y ~ 1)
```



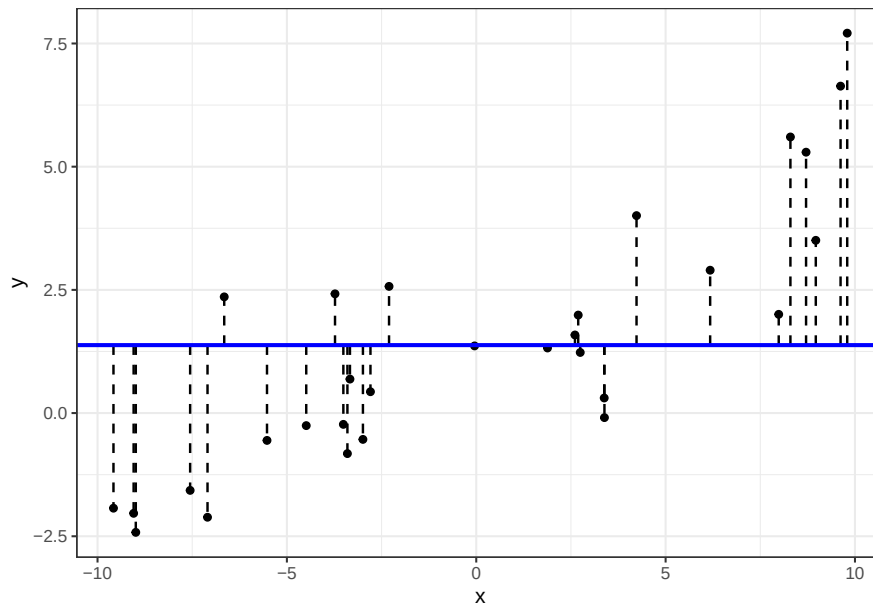
Notice also that in this case the intercept is simply equal to the mean of  $y$ :

```
mean(d$y)
```

```
## [1] 1.378907
```

How do residuals look for the intercept-only model? We can again use the `plot_resid` function for this purpose but this time we need to use the optional argument `slope` and set it to `FALSE`.

```
plot_resid(d$x, d$y, slope = FALSE)
```



As expected, it looks like the residuals are larger in the null-model. We can now calculate the sum of squares for the baseline model:

```
res_m0 <- residuals(m0)
sum_sq0 <- sum(res_m0^2)
```

Finally, we can express `sum_sq` as a fraction of `sum_sq0`:

```
sum_sq / sum_sq0
```

```
## [1] 0.3045986
```

This value tells us that our model has 30% of the left-over variation compared to the intercept-only model. If we instead subtract this value from 1, we get the fraction of variation our model is able to explain (again, relative to the intercept-only model):

```
1 - sum_sq / sum_sq0
```

```
## [1] 0.6954014
```

The above value is called the coefficient of determination (denoted as  $r^2$  or  $R^2$ ) and varies between 0 and 1. It is 0 when `sum_sq` and `sum_sq0` are equal, that is when the target has as much unexplained variation as the baseline model. It is 1 when `sum_sq` equals 0, that all points lie exactly on the regression line and all residuals equal 0.

Again, observe that this is identical to the **Multiple R-squared** value in the model summary (do not worry about the adjusted  $r^2$  for now, we will come back to it later).

Note also that in simple regression models (i.e. models with a single predictor), the coefficient of determination is equal the square of the correlation coefficient:

```
cor(d$x, d$y)^2
```

```
## [1] 0.6954014
```

However, this will no longer hold once we start adding more predictors to our models!

## Exercise

Calculate the  $r^2$  value for the `beginningReaders` model and compare it to the value in the model summary and to the square of the correlation coefficient. How do we interpret it?

*# Write your solution here.*

## Exercise

The file `f0_int.csv` includes fundamental frequency (`f0` in Hz) and sound pressure level (`spl`, in dB) values of a single speaker producing isolated vowels:

```
voice <- read_csv('f0_spl.csv')
head(voice)
```

```
## # A tibble: 6 x 2
##       f0    spl
##   <dbl> <dbl>
## 1  202.   72.3
## 2  197.   60.9
## 3  199.   55.2
## 4  199.   51.5
## 5  200.   50.5
## 6  203.   50.0
```

Fit a simple linear model predicting SPL based on fundamental frequency, plot the relationship and interpret the coefficients. In addition, calculate  $r^2$  and compare it with the value in the model summary.

*# Write your solution here.*

We will continue working with this model next time.